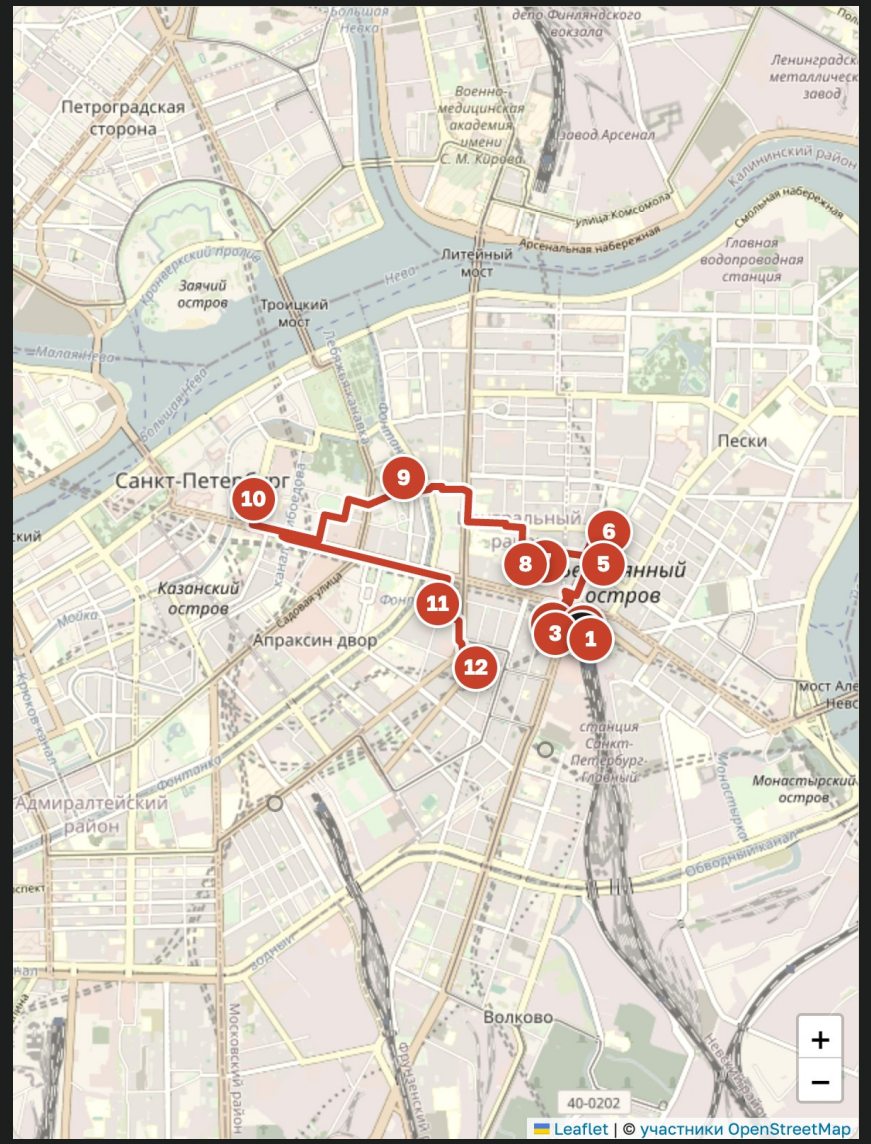
LLM AGENTS · НИУ ВШЭ · PROJECT PROPOSAL

Прогулка

LLM-агент строит пеший маршрут по городу из реальных мест — под интересы конкретного человека

1. **Роман Бекетов** @rbeketov
2. **Ярослав Кузьмин** @yarikTri
3. **Даниил Горнин** @dgornin
4. **Иван Прыгунов** @juzu400



Прогулка под себя — час ручной работы

Роман

Карты

где что находится и что рядом

Википедия, блоги, путеводители

что в этом месте интересного именно мне

Часы работы

успею ли я туда попасть

Пешком

сколько идти между точками и влезает ли всё в день

А если просто спросить LLM?

Та же модель, один промпт, 8 городов — проверка кодом

25 переходов, на которые времени меньше, чем идти (в 7 из 8 маршрутов)

4 места закрыты в момент визита

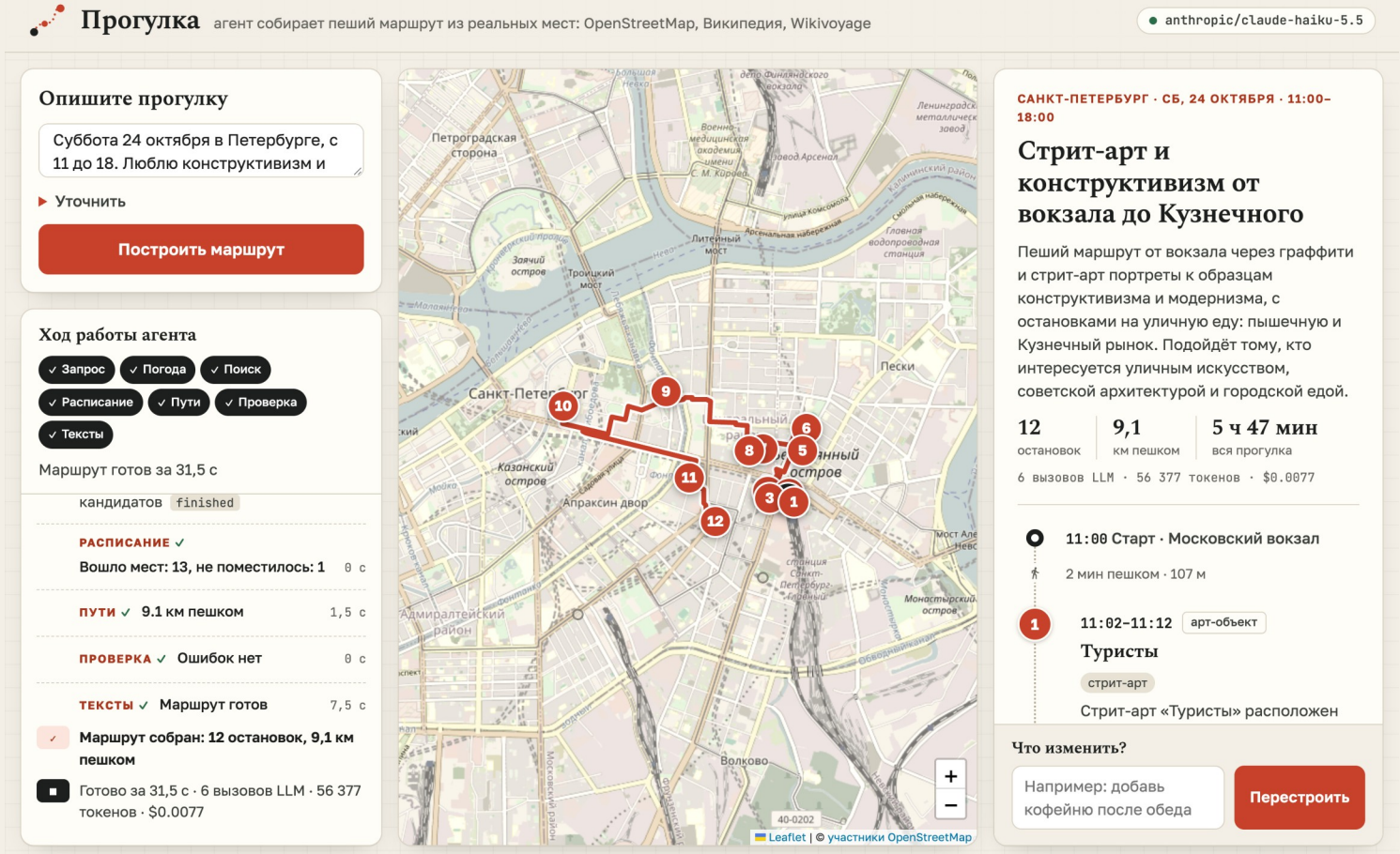
19 км в среднем пешком за «прогулку»

Запрос словами → проверенный маршрут

Роман

«Суббота в Петербурге, с 11 до 18. Люблю конструктивизм и стрит-арт, хочу уличную еду. Музеи не надо. Старт от Московского вокзала»

1. реальные места из OSM, Википедии и Wikivoyage — со ссылкой на источник
2. расписание: время, переходы пешком, часы работы
3. правка словами: «после 15 дождь» → маршрут перестроен



LLM — язык, код — факты и время

Роман

LLM

- понять свободный запрос: город, время, интересы с весами, что исключить
- решить, где искать каждый интерес: «конструктивизм» — Википедия, «стрит-арт» — объекты OSM
- отобрать места по их описаниям
- написать тексты и понять правку человека

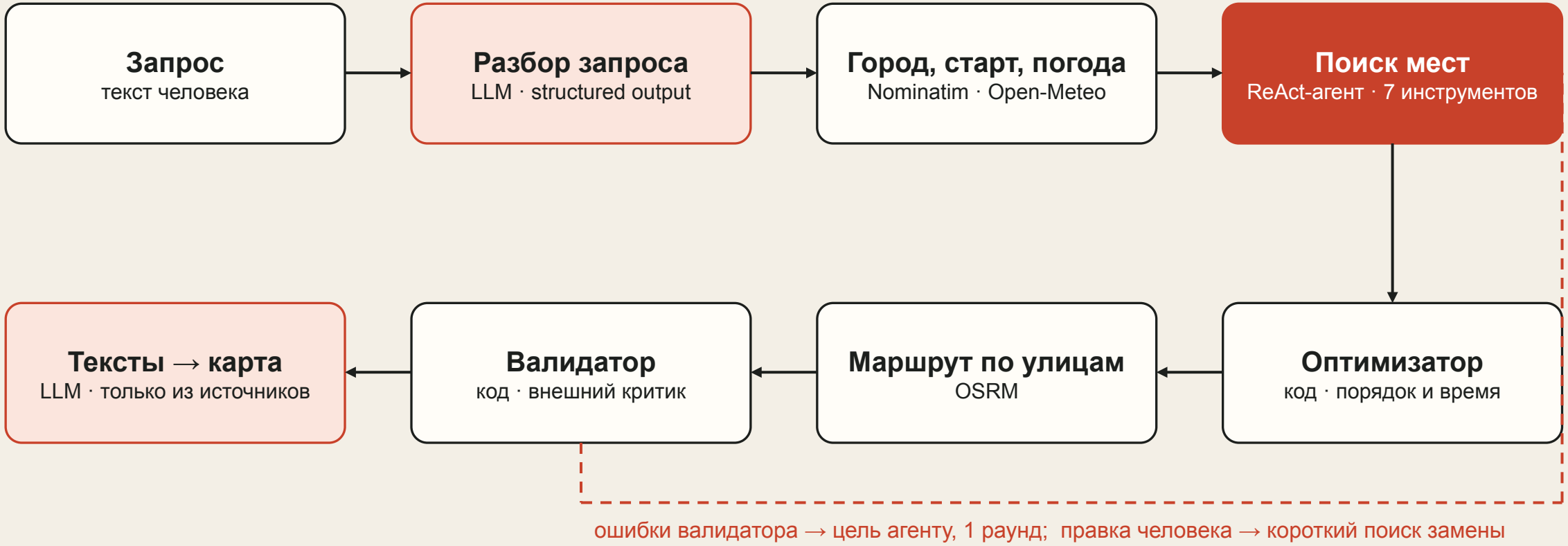
Код

- найти места, часы работы, погоду: OSM, Википедия, Wikivoyage, Open-Meteo
- выбрать, что успеть и в каком порядке: оптимизатор
- проверить план: открыто ли, влезает ли во время и в лимит км
- проложить путь по улицам: OSRM

Почему не «просто ChatGPT»: он делает и то, и другое сам — и ошибается именно в фактах, времени и расстояниях

Снаружи пайплайн, агент — только в поиске

Ярослав



Plan-and-Execute снаружи, ReAct внутри одного шага: только там следующий шаг зависит от найденного. Всё вычислимое — код.

Агентный цикл: свой, с явной остановкой

Ярослав

Инструменты (JSON-схемы, GET /api/tools)

```
search_osm(categories: enum[28], radius_m)
search_wikipedia(query | null, radius_m)
get_wikivoyage_listings(section)
get_place_details(place_id)
shortlist_place(place_id, score, interests, reason)
remove_place(place_id, reason)
finish_research(summary)
```

Grounding: в шорт-лист — только id, которые вернул инструмент. Категории OSM — enum: модель не пишет Overpass QL. Тексты источников — данные, а не инструкции.

Когда агент останавливается

`finished`	finish принят: мест хватает, сильные интересы покрыты
`max_steps`	14 шагов
`time_budget`	150 секунд
`repeated_action`	тот же вызов в 3-й раз
`no_tool_calls`	2 ответа подряд без инструментов
`llm_error`	провайдер не ответил — берём найденное

Причина остановки логируется и видна в интерфейсе

В среднем 7,0 вызовов LLM, 32 с и ≈ $0.010 на маршрут; параллельные tool calls

Порядок и проверка — детерминированный код

Иван

Оптимизатор

Задача ориентирования с временными окнами — NP-трудная, поэтому прозрачная эвристика:

- жадная вставка: ценность / добавленная минута
- 2-opt: короче путь при тех же правилах
- дождь: уличное ×0,4 в дождливые часы
- обед для прогулок от 4 часов
- убывающая ценность интереса ×0,7 — не десять муралов подряд
- лимит км по темпу; одинаковый вход — одинаковый маршрут

Валидатор

- закрыто в момент визита
- не укладывается во время
- то, что просили исключить
- уличное место под дождём
- длиннее лимита пешком
- интерес без единой остановки

Внешний критик: ошибки → цель для агента, ровно 1 раунд исправления. Сигнал извне, а не «модель, проверь себя»

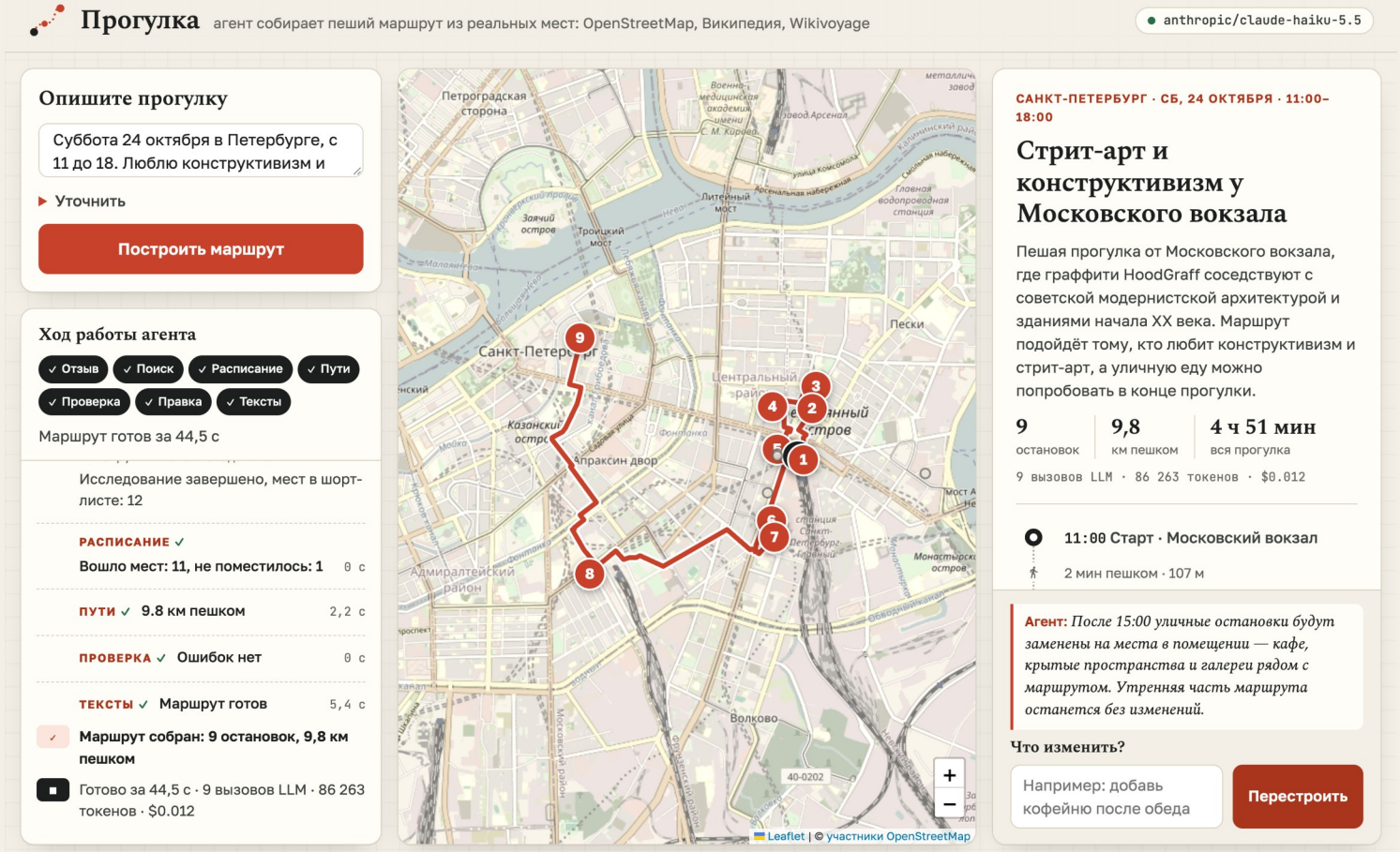
Источники: запрос к Overpass по bbox вместо around: — 1–3 с вместо таймаута; переключение между серверами; кэш ответов

Демо

Даниил

1. запрос про Петербург свободным текстом
2. агент ищет: места появляются на карте
3. маршрут: время, переходы, источники
4. «после 15:00 дождь» → перестроение

Запасной план: записанный прогон и LLM_REPLAY=1



Оценка: агент против одного промпта

Иван



Агент

77/77
мест из источников

0 / 0
закрыто / невозможных переходов

5,3 км
в среднем пешком

Один промпт

48/68
нашлись в OSM даже при мягком поиске

4 / 25
закрыто / невозможных переходов

19,0 км
в среднем пешком

8 запросов, 7 городов, одна модель (Claude Haiku 5.5), одна проверка кодом; места baseline ищем мягко

LLM: Claude Haiku 5.5 через OpenRouter

Даниил

Модель	Вызовов LLM	Время, с	Цена, $	Остановок	Ошибок
Claude Haiku 5.5	**8,0**	**38**	**0.0113**	**9,3**	**0**
GPT-6 luna	16,0	58	0.0036	8,7	0
DeepSeek v4.1 flash	10,0	57	0.0113	12,0	0
Nemotron 3 Super (free)	14,7	63	0.0000	4,7	0

Средние по 3 одинаковым запросам (СПб, Казань, Париж), полный пайплайн

- 4 роли: разбор запроса, агент с инструментами, тексты, правки
- меньше всего шагов и быстрее всех — важно для живого сценария
- надёжные tool calls и structured output на русском
- ≈ 1 цент за маршрут
- бесплатная модель не справилась: тратит все шаги и находит 2 места
- модель — одна переменная в .env: код от провайдера не зависит

К финалу: глубина, а не новые кнопки

Даниил

RAG по путеводителям

Wikivoyage и Википедия целиком; поиск мест «по смыслу», метрики retrieval

Память о путешественнике

темп, любимые темы, что уже видел — между сессиями

Большой eval

десятки запросов, LLM-as-judge для «попал ли в интересы», калибровка на ручной разметке

Деплой по ссылке

персистентные сессии, трассировка, учёт стоимости

Сознательно не делаем

- бронирование и оплату — необратимые действия без пользы для задачи
- мультиагентность ради мультиагентности — критик только если eval покажет выигрыш
- LangGraph, пока цикл помещается в 100 строк и чекпоинты не нужны

Спасибо!

Вопросы

github.com/dgornin/progulka

Роман Бекетов · Ярослав Кузьмин · Даниил Горнин · Иван Прыгунов

